Current Coding State

Current Coding State

Date: 2026-07-20

1. Project Structure

Thesis-specific code is contained in:

```
thesis_experiments/
```

Main modules:

```
thesis_experiments/
  adapters/
  canonical/
  data/
  datasets/
  evaluation/
  methods/
  scripts/
  tests/
```

External dataset storage is separated from the code repository.

2. Implemented Data Schema

The common RL trajectory schema is:

```
FootballTrajectory
```

Defined fields:

```
observations
actions
rewards
```

```
next_observations
terminated
truncated
episode_id
timesteps
player_positions
ball_positions
event_types
metadata
```

Implemented helpers:

```
concatenate_trajectories
split_trajectories
split_trajectories_by_match
```

3. Canonical Football Schema

Implemented canonical records:

```
CanonicalFootballEvent
CanonicalTrackingFrame
CanonicalSingleAgentState
CanonicalMultiAgentState
CanonicalMultiAgentAction
CanonicalMatch
CanonicalXY
```

These are used as the intermediate representation before creating RL trajectories.

4. Dataset Registry

Implemented:

```
data/dataset_sources.yaml
thesis_experiments/datasets/registry.py
```

Scripts can now use:

```
--from-registry
```

instead of manually passing dataset paths.

5. Dataset Converters

Implemented converters:

```
Dataset A   -> canonical events + tracking
Metrica     -> canonical events + tracking
StatsBomb   -> canonical events
Wyscout     -> canonical events
```

Implemented script:

```
python -m thesis_experiments.scripts.export_canonical
```

Generated canonical outputs:

```
dataset_a_bassek_2025_events.jsonl
dataset_a_bassek_2025_tracking.jsonl
metrica_sample_events.jsonl
metrica_sample_tracking.jsonl
statsbomb_open_data_events.jsonl
wyscout_events_events.jsonl
```

6. Action Mapping

Implemented central canonical action mapper:

```
thesis_experiments/canonical/actions.py
```

Current output action space:

```
GRF Discrete(19)
```

Mapped action labels include:

```
idle
long_pass
high_pass
short_pass
shot
sliding
dribble
release_dribble
movement directions
```

Implemented action inspection script:

```
python -m thesis_experiments.scripts.inspect_actions
```

7. Single-Agent State

Implemented single-agent state builder:

```
thesis_experiments/canonical/states.py
```

Single-agent state format:

```
shape: (115,)
type: GRF simple115-compatible vector
```

Implemented export script:

```
python -m thesis_experiments.scripts.export_states --mode single
```

Generated examples:

```
dataset_a_bassek_2025_single_states.jsonl
metrica_sample_single_states.jsonl
```

Validated:

```
Dataset A: 1000 states, observation length 115
Metrica:   1000 states, observation length 115
```

8. Multi-Agent State

Implemented multi-agent state builder:

```
thesis_experiments/canonical/states.py
```

Multi-agent state format:

```
shape: (22, 6)
features: x, y, vx, vy, controlled_team, active_player
```

Implemented export script:

```
python -m thesis_experiments.scripts.export_states --mode multi
```

Generated examples:

```
dataset_a_bassek_2025_multi_states.jsonl
metrica_sample_multi_states.jsonl
```

Validated:

```
Dataset A: 1000 states, feature shape (22, 6)
Metrica:   1000 states, feature shape (22, 6)
```

9. Multi-Agent Actions

Implemented:

```
thesis_experiments/canonical/multi_agent_actions.py
```

Multi-agent action format:

```
shape: (22,)
one GRF-style action id per tracked player
```

Implemented export script:

```
python -m thesis_experiments.scripts.export_multi_agent_actions
```

Generated examples:

```
dataset_a_bassek_2025_multi_agent_actions.jsonl
metrica_sample_multi_agent_actions.jsonl
```

Validated:

```
Dataset A: 999 frame transitions, 22 actions per frame
Metrica:   999 frame transitions, 22 actions per frame
```

10. RL Translation

Implemented canonical-to-RL builder:

```
thesis_experiments/canonical/rl_translation.py
```

Implemented script:

```
python -m thesis_experiments.scripts.build_rl_trajectories
```

Current implemented mode:

```
single-agent
```

Current grouping options:

```
match
possession
```

Generated RL samples:

```
dataset_a_canonical_rl_sample.pkl
metrica_canonical_rl_sample.pkl
```

Validated:

```
Dataset A sample: 1 trajectory, 11 transitions, observation shape (11,
115)
Metrica sample:   1 trajectory, 17 transitions, observation shape (17,
115)
```

11. Train/Validation/Test Splitting

Implemented split script:

```
python -m thesis_experiments.scripts.split_trajectories
```

Implemented split type:

```
match-level split
```

Current bounded Dataset A split:

```
train:      5 matches, 2495 transitions
validation: 1 match,  499 transitions
test:       1 match,  499 transitions
```

Generated files:

```
dataset_a_7match_event_rl_500_train.pkl
dataset_a_7match_event_rl_500_validation.pkl
dataset_a_7match_event_rl_500_test.pkl
```

12. Offline Summary

Implemented summary script:

```
python -m thesis_experiments.scripts.evaluate_offline
```

Outputs:

```
number of trajectories
number of matches
number of transitions
mean/min/max trajectory length
reward sum
nonzero reward count
action distribution
top event types
```

Generated summaries:

```
dataset_a_7match_event_rl_500_summary.json
dataset_a_7match_event_rl_500_train_summary.json
dataset_a_7match_event_rl_500_validation_summary.json
dataset_a_7match_event_rl_500_test_summary.json
```

13. Environment Wrapper

Implemented:

```
thesis_experiments/env.py
```

Current environment setup:

```
GRF single-agent
legacy Gym API
simple115-style observation
Discrete(19) action space
```

The wrapper handles:

```
environment creation
seeding
Gym/Gymnasium compatibility where needed
```

14. Method Adapters

Implemented adapter interface:

```
thesis_experiments/adapters/base.py
```

Implemented adapter files:

```
bc.py
gail.py
```

```
ppo.py
cql.py
dt.py
ghosting.py
```

Current status:

```
BC: implemented smoke path
PPO: implemented smoke path
GAIL: skeleton / smoke-oriented integration
CQL: skeleton, needs separate d3rlpy environment
DT: placeholder adapter, real backend not integrated
Ghosting: small PyTorch baseline for interface validation
```

15. Smoke Tests

Implemented:

```
python -m thesis_experiments.scripts.smoke_test --method bc
python -m thesis_experiments.scripts.smoke_test --method ppo
python -m thesis_experiments.scripts.smoke_test --method gail
python -m thesis_experiments.scripts.smoke_test --method cql
python -m thesis_experiments.scripts.smoke_test --method dt
python -m thesis_experiments.scripts.smoke_test --method ghosting
```

Verified locally:

```
BC smoke passed on Dataset A train split
DT placeholder smoke passed on Dataset A train split
Dataset converters compile
canonical converters compile
state/action/RL translation tests pass by direct execution
```

16. Tests Added

Implemented tests:

```
test_converters.py
test_dataset_a.py
test_canonical.py
test_canonical_actions.py
test_multi_agent_actions.py
test_canonical_states.py
```

```
test_rl_translation.py
test_splits.py
```

Current note:

```
pytest is not installed in the current environment.
Tests were executed directly as Python functions.
```

17. Dependency Status

Current main environment supports:

```
GRF
gym 0.21
stable-baselines3
imitation
torch
gymnasium compatibility utilities
```

CQL status:

```
d3rlpy is not installed in the main GRF environment.
It should be tested in a separate environment.
```

Reason:

```
d3rlpy dependency versions conflict with the GRF gym dependency.
```

18. Current Working Pipeline

The current working single-agent coding pipeline is:

```
Dataset A
    ->
canonical events/tracking
    ->
single-agent states/actions/rewards
    ->
FootballTrajectory
    ->
match-level train/validation/test split
    ->
BC and DT smoke tests
```

Current broader data pipeline:

```
Dataset A   -> canonical events + tracking -> RL trajectories
Metrica     -> canonical events + tracking -> RL samples
StatsBomb   -> canonical events
Wyscout     -> canonical events
```

19. Current Missing Implementation

Still incomplete:

```
full evaluation protocol
full-scale dataset conversion without caps
CQL environment separation and CQL smoke run
GAIL full compatibility check
Decision Transformer real backend
multi-agent RL training backend
reward validation for goals/outcomes
PPO baseline evaluation
final experiment runner
standard result table generation
```

20. Immediate Coding Next Steps

Recommended next coding tasks:

```
1. Add pytest to development dependencies.
2. Create Linux/VM setup script.
3. Add CQL-specific environment instructions.
4. Run CQL from processed FootballTrajectory files.
5. Replace DT placeholder with a real PyTorch backend.
6. Verify GAIL with vectorized GRF on Linux.
7. Add a common experiment runner.
8. Add common evaluation result JSON/CSV output.
9. Run full Dataset A conversion without caps.
10. Add multi-agent trainer prototype.
```